

LASER: Script Execution by Autonomous Agents for On-demand Traffic Simulation

Hao Gao
gh@smail.nju.edu.cn
State Key Lab of Novel Software
Technology, Nanjing University
China

Jingwei Xu
jingweix@nju.edu.cn
State Key Lab of Novel Software
Technology, Nanjing University
China

Jingyue Wang
jingyuew@smail.nju.edu.cn
State Key Lab of Novel Software
Technology, Nanjing University
China

Yunpeng Huang
hyp@smail.nju.edu.cn
State Key Lab of Novel Software
Technology, Nanjing University
China

Wenyang Fang
211220178@smail.nju.edu.cn
State Key Lab of Novel Software
Technology, Nanjing University
China

Taolue Chen
taolue.chen@gmail.com
School of Computing and
Mathematical Sciences, Birkbeck,
University of London
UK

Xiaoxing Ma
xxm@nju.edu.cn
State Key Lab of Novel Software
Technology, Nanjing University
China

Abstract

Autonomous Driving Systems (ADS) are advancing rapidly due to progress in deep learning, yet critical challenges remain, particularly in the realm of safety verification. As safety-critical systems, ADS must undergo rigorous testing across diverse scenarios. Real-world data, while valuable, are inherently inflexible for interaction and scenario customization. In contrast, simulator-generated synthetic scenarios provide a platform that enables interaction, control, editability, and adaptability to specific needs. However, current simulation approaches are limited—either relying on costly, manually crafted, overly templated scenarios or generating unconditioned trivial behaviors based on learned distributions.

In this work, we introduce LASER, an innovative framework that leverages large language models (LLMs) to conduct traffic simulations based on natural language inputs. The framework operates in two phases. First, it generates scripts from user-provided descriptions. Second, it executes these scripts by guiding autonomous agents within the CARLA simulator to perform tasks in real-time. This method effectively decomposes tasks, allocates controls, and integrates interactive elements to create dynamic and scalable simulations that align with user requirements. By using LASER, we overcome the rigid constraints of traditional simulation methods, enabling the creation of complex, diverse, flexible and on-demand driving scenarios. The approach significantly enhances the process of generating ADS training and testing data, addressing the scalability and diversity issues associated with previous simulation models. The code and all demos are available anonymously at <https://njudeepengine.github.io/LASER/>.

Keywords

Autonomous Driving Systems, Traffic Simulation, Autonomous Agents, Test Generation

To make a great film, you need three things—the script, the script and the script.

– Alfred Hitchcock

1 Introduction

With the deep learning breakthrough, Autonomous Driving Systems (ADS) have made significant advancements in tasks such as occupancy prediction [15, 35], trajectory prediction [13, 20, 29], semantic scene completion [5, 16, 17] and world model [31, 32, 37, 43]. For example, Tesla’s Full Self-Driving (FSD) demonstrates exceptional performance in common and complex tasks such as lane change, turn, merge, fork, and detour for most types of roads, including even curvy highways and roundabouts [28].

The rapid development of end-to-end ADS research and application is based on vast and high-quality driving data. On the one hand, a large dataset containing multimodal data from all kinds of sensors (such as cameras, lidar, and radar) is essential to train the underlying deep neural network (DNN) models. Taking FSD as an example, Tesla claims that their camera-only models are trained with more than 160 billion frames of driving data sampled from real-world scenarios, synthetic scenarios generated by simulators, as well as those from other sources [27]. On the other hand, ADS are highly critical in safety and must be thoroughly tested in various scenarios, especially some rare and unanticipated ones, to guarantee their ability to handle emergence and avoid accidents [38].

The most straightforward approach to building datasets for ADS training/testing is to collect real-world traffic data through sensors such as vehicle cameras, which naturally reflect real distributions of the data and can be scaled up through crowd-sourcing [3, 4]. However, this method is inefficient, as daily traffic often yields repetitive, trivial scenarios, while safety-critical events, which are rare and high-risk, are seldom included in the training set and thus hardly learned by the model [12]. Additionally, static data, such

as vehicles captured in each frame, lack the flexibility to interact with or manipulate, preventing effective training/testing specific or customized scenarios. Last but not least, online, interactive testing of ADS requires the actors (e.g., vehicles, pedestrians) to be reactive to the behavior of others, which is virtually infeasible for those collected from traffic data.

To address these limitations, another class of approaches is to generate the scenarios from traffic simulators, collecting synthetic data through high-fidelity sensors [3, 4, 22, 26, 34, 41, 42]. These methods allow for creating customized driving scenarios tailored to specific needs, resulting in a controllable and editable dataset for training and testing models. Furthermore, they enable the rapid execution of thousands of diverse and targeted online tests by deploying virtual vehicles attached with trained models in the simulator, facilitating the identification of pitfalls and the resolution of exceptions before costly real-world settings.

Undoubtedly, generating driving scenarios with real-world traffic flows is demanding. Each dynamic object—regardless of a vehicle, bicycle, or pedestrian—exhibits its own time-varying motion patterns, which are often interdependent with those of other objects. Mainstream methods for traffic simulation can be categorized as *rule-based* or *learning-based*. Rule-based traffic simulation employs analytical models to control vehicle movements [6, 11, 18], typically relying on fixed, predefined routes. This approach often results in highly repetitive scenarios with limited behavioral diversity.

In contrast, learning-based methods aim to replicate human trajectories from real-world driving logs to produce varied and realistic behaviors, which leverage techniques—such as imitation learning (IL), reinforcement learning (RL), deep learning (DL) and deep generative models—to generate diverse and realistic driving behaviors by utilizing real-world driving logs as demonstrations. We refer the readers to [7] for a comprehensive survey. However, these methods generally face significant challenges in accurately modeling and generating human driving behaviors, often resulting in simplistic actions such as passing or merging [26]. This is primarily due to three reasons. (a) **Limited and biased training data.** Existing methods commonly rely on datasets such as Nuscenes, which contains only 1000 videos [3]. Learning high-level driving behaviors in complex, multi-agent environments must tackle the combinatorial explosion of input states, but the limited, imbalanced nature of the data makes it hard to generalize to rare or unseen scenarios, commonly referred to as “long-tail” cases [7]. (b) **Lack of alignment with human understanding.** The behaviors generated by these models are often not aligned with natural language descriptions or human common sense, making them less interpretable and harder to customize for specific driving behaviors. (c) **Scenario generation in real-time.** When generating interactive traffic scenarios online, these methods typically operate in an auto-regressive manner, where each step’s prediction builds upon the previous one. Without goal-oriented guidance, this approach can lead to trivial or ineffective behaviors, and the accumulation of prediction errors may result in catastrophic failures such as collisions or vehicles driving off-road [40].

Very recently, deep generative model-based methods [31, 32, 37, 43] provide a promising way for generating customized traffic data with world models. However, it remains a difficult task to generate *interactive* traffic scenarios with diverse, on-demand behaviors.

Recently, large language models (LLMs) and multi-modal language models (MMLMs) have demonstrated remarkable capabilities in common-sense reasoning, planning, interaction and decision-making, showcasing great potential to address the challenges mentioned above [10, 14]. We propose a new traffic simulation framework, named **LASER (LLM-based scenArio Script gEerator and ExecutoR)** that leverages LLMs to create both intricate and interactive driving scenarios by generating readable scripts to guide step-by-step execution of each dynamic objects within the scenarios, which only requires simple natural language descriptions from the users in the first place.

As illustrated in Figure 1, we first translate user requirements to a master script, which then is converted to sub-scripts for each dynamic object, i.e., each executing actor in the scenario. Based on the rich domain-specific knowledge and advanced reasoning capability of LLM4AD [23, 36], each actor’s lifespan is managed by an LLM-controlled autonomous agent that executes its sub-script in real-time. These agents make decisions about intermediate actions based on the current state of the environment at each simulation timestamp, all aiming to achieve their individual goals within the expected time frame.

Leveraging the common-sense and behavior understanding of LLM-controlled agents, we can perform top-down behavior-to-action script interpretation, as opposed to the previous bottom-up action-to-behavior accumulation. The LLM interpretation aligns language-specified behaviors with low-level actions, provides interpretability for the generation process and enables the on-demand generation of specific long-tail scenarios. With scripts highlighting their agendas, these LLM-controlled autonomous agents cooperate to achieve the tasks, generating on-demand behavior while avoiding accumulated prediction errors.

We design a task set consisting of 17 user requirements encompassing both long-tail and reasonable safety-critical scenarios. We evaluate LASER for on-demand script generation and execution on these tasks in the CARLA simulator [8]. The experimental results demonstrate that LASER can generate scripts based on user requirements effectively, with only 3.18% of the characters in the resulting executable script being inputted by the user to fulfill their demands. The experimental results also show that our approach can execute the script effectively and efficiently, with an average success rate of 90.48%, and usage of 1606.09 tokens per simulation second per agent. Furthermore, manual inspection confirms that our approach can successfully simulate various safety-critical scenarios which can be applied to ADS testing.

In summary, the primary contribution of our work is to propose an on-demand, interactive approach for traffic simulation, which includes a script generator and LLM-controlled autonomous agents as the executor. To the best of our knowledge, this is the first time we have achieved on-demand scenario generation in ADS testing.

2 Related works

We have covered the related work on learning-based traffic simulation in Section 1. A recent trend in autonomous driving is to leverage LLMs which have demonstrated exceptional capabilities in human-like tasks such as common-sense understanding, planning, decision-making, and interaction [10, 14], trained on vast

datasets of trillions of tokens and images from the web. These models exhibit a deep reservoir of actionable knowledge, which can be harnessed for robotic manipulation through reasoning and planning [36]. Recent research has explored the use of LLMs to develop autonomous agents that execute natural language tasks in interactive environments [1, 2, 9]. A notable example is RT-H, which enhances agent robustness and flexibility by decomposing high-level tasks into sequences of fine-grained behaviors, referred to as "language motions" (e.g., "move arm forward" followed by "grasp the can"). This approach effectively leverages multi-task datasets, significantly improving performance [1].

Substantial efforts have also been directed towards integrating LLMs with ADS, underscoring the models' superior abilities in understanding and decision-making within driving scenarios [21, 23, 25, 36]. For example, CarLLaVA achieved first place in the sensor track of the CARLA Autonomous Driving Challenge 2.0, surpassing the next-best submission by 32.6%. This success is attributed to its integration of the vision encoder LLaVA with the LLaMA architecture as its backbone [21]. Additionally, LLM-Assist outperformed all existing learning- and rule-based methods across most metrics in the Nuplan dataset by leveraging LLMs' common-sense reasoning to refine plans generated by rule-based planners [25].

Further research has demonstrated the capability of LLM-integrated ADS to execute tasks based on natural language instructions, revealing its potential for modeling complex human driving behaviors [23, 30]. For instance, LMDrive showed that LLM-controlled driving agents could interpret and follow high-level driving commands, such as "Turn right at the next intersection," by aligning these instructions with vehicle control signals using a vision-language model as the foundation [23]. More recently, DiLu demonstrated that with few-shot learning, LLMs could achieve results comparable to RL-based planners, significantly reducing the computational cost of deploying multiple LLM-controlled agents simultaneously [36].

3 Methodology

3.1 Framework

To achieve on-demand and interactive traffic simulation, we propose a framework called **LASER**, as illustrated in Figure 1. LASER consists of two stages, implemented by two modules respectively, i.e., script writer (Section 3.2) and LASER-Agent (Section 3.3). Unlike the previous learning-based methods that conduct generation and simulation simultaneously, our LASER framework first generates scripts that define logic-chained behaviors (LCB) with natural language instructions from the user requirements. It then executes the script by the real-time cooperation of LASER-Agents.

The two-stage framework detaches actor behavior from the scenario with a natural language script. Compared to road data collection that collects on-road scenarios at the state level, our LASER framework records scenarios at the behavioral level, which enables dynamic and flexible execution during the simulation. Compared to learning-based traffic simulation that generates scenarios at the state level, LASER generates scenarios at the behavioral level, this enables on-demand generation and top-down execution of language-specified behavior, and easy editing on actor behavior to improve performance.

To have a glance at the complexity of behaviors that LASER can simulate, we present an example of a user-defined scenario in Figure 1. The user requests a safety-critical scenario on the highway: Two cars are driving in the left lane ahead of the ego vehicle, which is in the right lane. Suddenly, the front car decelerates due to a mechanical failure. The rear car, unable to brake in time, switches to the right lane to avoid colliding with the front car, unaware that the ego vehicle is behind it. In this situation, the ego is responsible for quickly recognizing the potential for the rear car to change lanes due to the front car's abrupt deceleration and must react by decelerating promptly to prevent a collision. In the following sections, we will demonstrate detailed implementations of the script writer and LASER-Agent modules with the example shown in Figure 1.

3.2 Script writer

Directly executing user requirements with autonomous agents is unstable. User requirements can be ambiguous, leading to varying behaviors across different execution attempts. Additionally, fulfilling the user requirement often requires coordination among multiple autonomous agents. Without a shared consensus on how to achieve this goal, these agents may act independently, potentially resulting in failure.

To address these challenges, we use script writer to translate user requirements into scripts. To ensure consistency across different execution attempts, the behaviors in the script must be detailed and concrete, with sub-scripts that contain detailed LCB instructions for each agent to execute. To ensure effective coordination, these sub-scripts must align under a single master script, serving as a unified consensus for all agents.

In our work, the generation of scripts follows a hierarchical chain-of-thought (CoT) manner [33] to enhance the common-sense reasoning and planning ability of LLM as GPT-4o does [19]. The script writer first generates a master script that outlines the framework of the story, then further generates sub-scripts that contain detailed LCB instructions for each actor based on the master script. This ensures behavior consistency over agents and executions. The scripts are written in natural language, to enable easy editing of the behaviors. The generating procedure of the master script and sub-scripts for individual agents is shown in the following paragraphs.

Master-script Generation. Based on the user's initial requirements and (optionally) a map description to specify the surrounding layout, the script writer generates a master script using a CoT approach. First, the script writer prompts the LLM to generate a story that aligns with the user's requirements. The LLM then outlines key stages in the sequence of events, starting with the initial state, where each stage acts as a direct cause or prerequisite for the next. For example, given the user requirements illustrated in Figure 1, the following master script is produced:

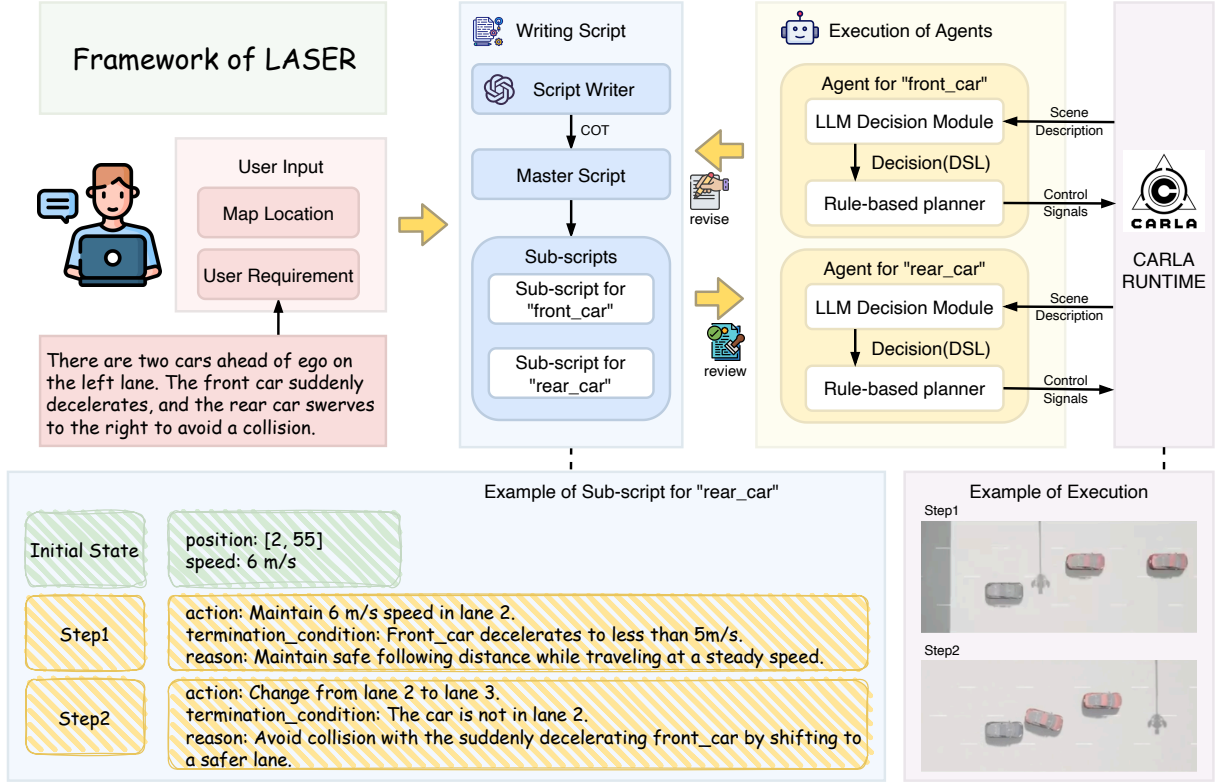


Figure 1: Framework of LASER

- (1) Initial state: The Vehicle Under Test (VUT) is in the rightmost lane, with two cars ahead in the leftmost lane.
- (2) Stage 1: The front car in the leftmost lane suddenly decelerates. Reasoning: The deceleration may be due to an obstacle or the need to reduce speed significantly for an intersection.
- (3) Stage 2: The rear car in the leftmost lane swerves into the rightmost lane. Reasoning: The rear car swerves to avoid a collision with the front car.

Sub-scripts Generation. Based on the master script, the script writer queries the LLM to generate sub-scripts containing detailed, step-by-step LCB instructions for each individual actor. These sub-scripts clearly outline the specific actions each actor must perform, using natural language to chain behaviors logically. Each action is paired with a termination condition, specifying when the task is complete and when the next action should begin, along with the reasoning behind it. This ensures that the behavior not only aligns with the overall narrative but also allows for flexible execution by the autonomous agent.

The resulting sub-scripts are structured similarly to movie scripts. They begin with an initial state, defining the actor type (e.g., truck, car, or pedestrian), and then break down into several sequential steps. Each step includes an action, termination conditions, and a

reason, all of which work together to link the actor's behavior with logical decision-making. An example of a sub-script is shown in Figure 1. The initial state specifies the actor's lateral and longitudinal position, as well as its initial speed. The action defines a simple, concrete motion that can be easily executed by our LASER-Agent, such as merging into the leftmost lane. The termination condition outlines measurable criteria, which may depend on the actor's own behavior or interactions with others, for example, when the longitudinal distance to another vehicle is within 2 meters. Finally, the reason clarifies the rationale behind each action, enhancing the agent's understanding and enabling more flexible execution, especially during interactions with other actors.

3.3 LASER-Agent

The second stage of grounding the sub-scripts into execution is achieved through the collaboration of LASER-Agents. We employ LLM-controlled driving agents to facilitate the comprehension of language instructions and behavior. These agents, designed to operate autonomously, execute the sub-scripts step by step based on real-time environmental observations, working together to bring the entire scenario to life. Since fine-tuning LLMs with vehicle control signals and applying LLMs to learning-based planners both require substantial computational resources during runtime (especially when managing multiple agents), we integrate each agent with an LLM-based decision module alongside a rule-based planner. The LLM-based decision module, equipped with common-sense,

plays a crucial role in converting language-based LCB instructions into executable actions. Every 0.5 seconds, the LASER-Agent encodes the environmental scenario into a descriptive format, integrates sub-script LCB instructions to create a prompt, and queries the LLM for an executable decision. The rule-based planner then carries out this decision.

LLM-based Decision Module. This module processes the scenario description along with LCB sub-script instructions, generating an executable decision every 0.5 seconds. It begins by checking if the termination condition for the current step is met. If the step is complete, it transitions to the next one. The module then predicts an executable decision based on the step’s instructions and the scenario description, which includes parameters such as target speed, lane change direction and lane change delay.

LLM brings a common-sense understanding of translating language instructions into higher-level executable decisions defined by a DSL, including `lane_change_direction` and `target_speed`. Instead of directly outputting vehicle control signals where LLMs do not excel, this higher-level decision-making process offers better alignment with the model. This approach enables zero-shot grounding of language-specified behaviors more effectively.

To enhance LLM’s comprehension of the current traffic environment, we encode the surrounding traffic conditions into a standardized textual scenario description, following the approach outlined in DiLu [36]. This description includes all essential information for decision-making, such as the number of available lanes, the positions, speeds and lane-change statuses of both the subject vehicle and surrounding vehicles.

Rule-based Planner. It outputs vehicle control signals to execute the decisions made by the LLM-based decision module at every frame. It tracks a path consisting of waypoints on the map and uses PID control [39] to regulate speed. Whenever the rule-based planner receives a new decision from the LLM-based decision module, it generates a new path based on the current position, following the lane change direction and the delay specified in the decision. At each frame, the planner tracks the waypoints on the path, while the PID controller calculates the vehicle’s steering and throttle.

This lightweight design allows the control of multiple agents simultaneously, enabling their complex behaviors and interactions. The rule-based planner functions as a humble executor of the LLM’s decisions, without incorporating safety constraints such as maintaining distance from other vehicles. This design ensures that the agent can exhibit alarmingly realistic behaviors.

3.4 Connecting with CARLA

To enable real-time dynamic interaction between the LASER-Agent and the CARLA environment, we develop a runtime system that allows the agent to both perceive and operate within the environment, as shown in Figure 2.

For perception, every 0.5 seconds, we acquire key information about each vehicle, including its location, speed, and acceleration. This data is encoded and provided to each vehicle, where waypoints are used to convert global coordinates into Frenet coordinates along the lane. Additionally, we refine the representation into a more LLM-friendly format, incorporating natural language expressions

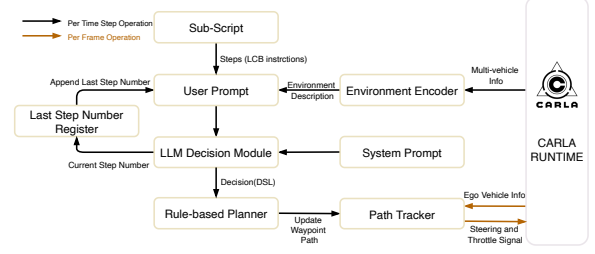


Figure 2: Connection with CARLA

such as “ahead of you” or “driving in the same lane” to enhance interpretability.

For operation, we leverage CARLA’s waypoint system to navigate through lanes and intersections. In CARLA, waypoints define lane centers and connect lanes at intersections, enabling multiple routing options. The rule-based planner acquires these waypoints and constructs paths based on them when given a new decision from the LLM-based decision module.

To ensure fine-grained and precise control, a tracker continuously generates steering and throttle commands, guiding the vehicle along the waypoint path in each frame of the simulation. This allows for smoother and more accurate maneuvering of the LASER-Agent.

4 Evaluation

In this section, we present a comprehensive evaluation of script writer and LASER-Agents for on-demand traffic simulation. All code and the results are available on one repository.¹

4.1 Setting

User requirements. We design 17 scenario generation tasks, each representing a complex traffic execution requirement that is challenging to capture through traditional road collection or existing simulation methods (cf. Table 1 for details). For example, tasks such as “Accident”, “Ambulance”, and “Reckless Driving” present long-tail scenarios that are rarely encountered. “Swerve” and “Three in Line 1” depict reasonable safety-critical situations where even experienced drivers could make mistakes. To navigate these scenarios effectively, one must be able to anticipate signs of an impending accident.

LASER Setup. Our experiment utilizes CARLA 0.9.15 [8], a widely used open-source simulator for closed-loop ADS testing. Built on the UE4 engine, CARLA offers realistic graphics, a variety of vehicle and pedestrian models, and diverse maps. We assess each task across three road segments from Town04, Town05, Town06 and Town10, conducting 20 simulations per segment for effectiveness (totaling 60 simulations per task) and 5 simulations per segment for efficiency (totaling 15 simulations per task).

All experiments employ GPT-4o as the LLM, which queries every 0.5 seconds. To evaluate interactions between LASER-Agents and ADS in safety-critical scenarios, we use our LASER-Agents to test the end-to-end ADS InterFuser [24], which ranked #1 in the CARLA challenge 2022 among the open source models.

¹<https://njudeepengine.github.io/LASER/>

Table 1: Task set (* indicates safety-critical tasks)

Task	User requirement
1. Accident	An accident occurred on the highway. One car turned left to avoid it, but with the left lane occupied, the following car turned right.
2. Ambulance	On the highway, an ambulance is driving straight at high speed. Multiple vehicles in the same lane move to both sides to make way for it.
3. Bus	A bus switched from the middle lane to the bus stop, then started again and changed lanes back.
4. Caught in Pincer	A car to the left is overtaking, forcing the ego vehicle to decelerate, while a car behind speeds up, pressuring it to accelerate.
5. Cut-in*	A car overtakes the VUT and then slows down.
6. Failed at Start	A car parked in front of the bus station started, changed lanes, and collided with the bus that was changing lanes to park.
7. Meet in Mind 1	Two cars simultaneously change lanes to the middle lane.
8. Meet in Mind 2	Two cars, unable to see each other, simultaneously attempt to overtake a car in the middle.
9. Merge Alternately	Two cars collided in the right lane at a road intersection, and several cars in the rear weaved into the left lane.
10. Newbie Lane Change 1	A car in front on the right changes lanes without accelerating, causing a collision with the ego vehicle.
11. Newbie Lane Change 2	A truck is to the left of the ego vehicle, and a car in front on the right changes lanes without accelerating, causing a collision with the ego vehicle.
12. Reckless Driving	A reckless driver repeatedly overtakes other cars on the highway.
13. Sudden Jaywalker*	The VUT was moving in the left lane behind a stopped truck. As it approached, a pedestrian emerged from behind the truck.
14. Surrounded	Four police cars surround the criminal’s vehicle from the front, back, left, and right.
15. Swerve*	There are two cars ahead of ego on the left lane. The front car suddenly decelerates, and the rear car swerves to the right to avoid a collision.
16. Three in Line 1*	There are two cars in front of the VUT. The car at the very front slows down, and the second car, to avoid a collision, changes lanes to the right.
17. Three in Line 2	The front car suddenly hit the stopped car ahead, and the ego car collided with the front car.

Road Segments. We evaluate each task on three highway segments (including one curved segment) and three urban segments (including a curved road). Highways typically feature longer, uninterrupted stretches with multiple lanes, while urban roads often have more frequent intersections, narrower lanes, and varying road geometries. Straight and curved road segments also present distinct challenges, with straight roads focusing on maintaining lane discipline and curved segments requiring more dynamic path adjustment. This variety ensures the generalizability of LASER across different road configurations.

Metrics. The effectiveness of script writer is measured by the user involvement percentage, defined as the average proportion of user-provided characters (excluding spaces and newlines) in the final executable script. We assess script execution success rate as the number of traffic simulations meeting user requirements divided by the total number of simulations conducted. Generated traffic simulations are manually reviewed against specific criteria. For example, in the scenario “Accident”, the criterion requires the car to successfully overtake the front car and subsequently slow down. In another scenario “Three in Line 1”, the criterion specifies that the front vehicle must decelerate to a stop, while the rear vehicle changes lanes to the right before colliding with the front vehicle and then drives away. Additional criteria are available on our project repository for further reference. Efficiency is evaluated using token cost, defined as the number of tokens used per simulation second

per agent, and time cost, representing the real-world simulation time per simulation second.

4.2 Evaluation on script generation

To evaluate the effectiveness of on-demand script generation, we select eight user requirements related to long-tailed scenarios from the task set. For each task, script writer generates five scripts, which we then manually refine until the scripts successfully fulfill the user requirements.

The experimental results are summarized in Table 2, while Figure 3 visualizes a selected case for the task “Swerve”. The results indicate that script writer effectively generates on-demand scripts, with an average user involvement percentage of just 3.18%. Most inaccuracies in script writer’s outputs stem from imprecise numerical values, such as positions and speeds. Additionally, there are instances where script writer overlooks steps implied by the user requirements. For example, in the “Failed at Start” task illustrated in Figure 4, the bus should initially move in the left lane while the car stops in the right lane, before both vehicles change lanes simultaneously. However, script writer incorrectly bypasses this step, causing the bus to change lanes at the start.

4.3 Evaluation on script execution

To evaluate the effectiveness and efficiency of script execution, we select eight user requirements from our task set that involve

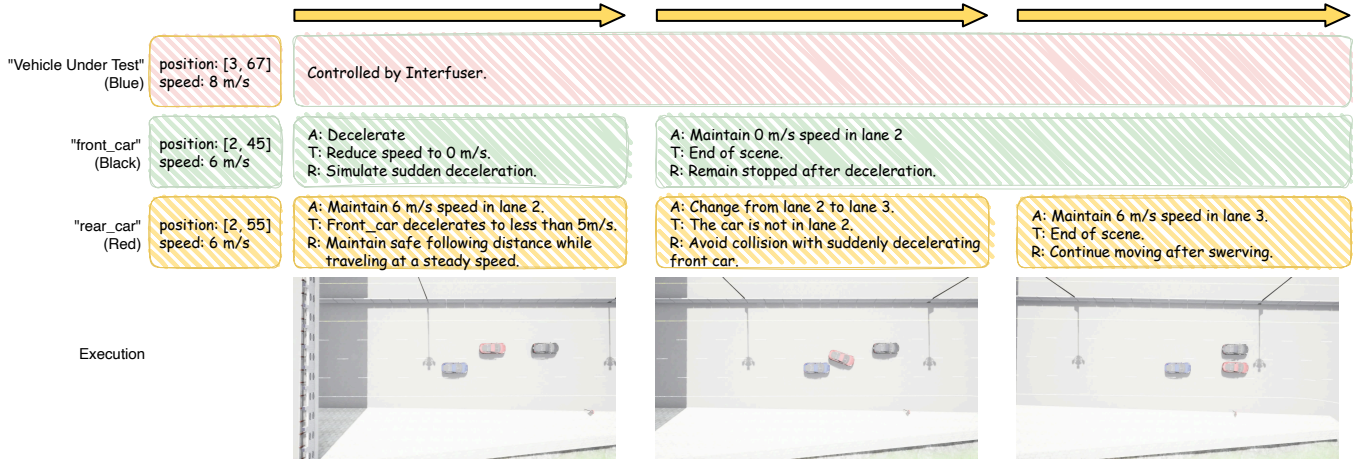


Figure 3: A case of script generation result. (A: Action T: Termination Condition R: Reason)

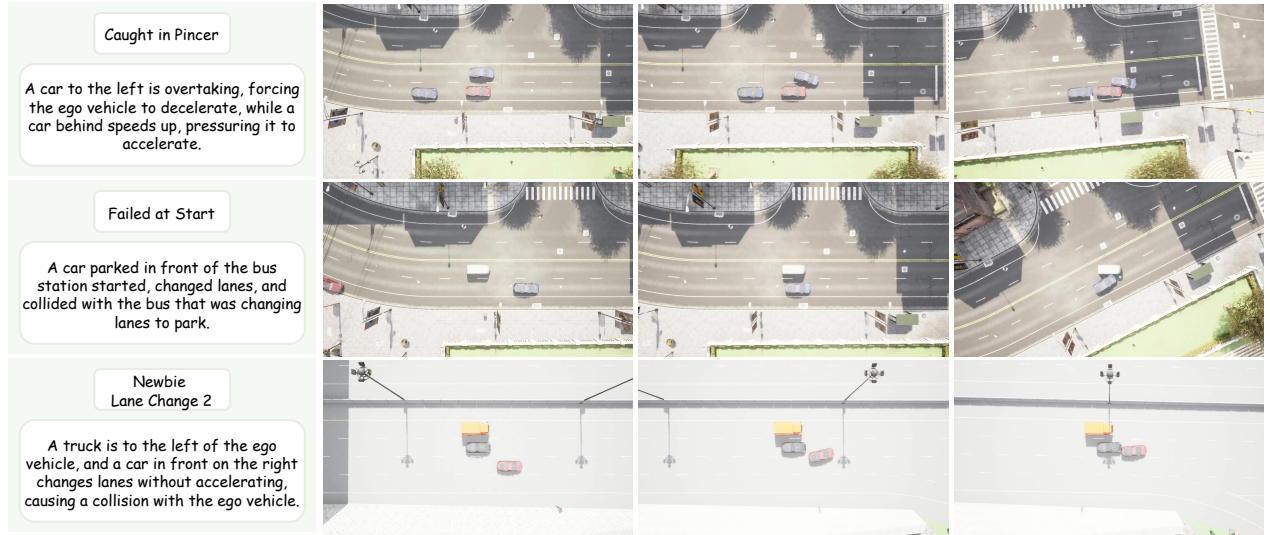


Figure 4: The visualized results for script generation.

Table 2: Evaluation of user involvement in script generation. (* indicate safety-critical tasks)

Task	User involvement percentage ↓
Accident	5.43%
Ambulance	4.41%
Caught in Pincer	0.47%
Failed at Start	7.36%
Newbie Lane Change 2	0.35%
Cut-in*	1.50%
Swerve*	3.15%
Three in Line 1*	2.74%
average	3.18%

complex interactions, including four tasks focused on safety-critical scenarios. For each task, script writer generates a script based on the user requirement, which we then manually modify to ensure it meets the requirements fully. Scripts are executed by our LASER-Agents to assess their effectiveness and efficiency.

Effectiveness. The experimental results are shown in Table 3. The results indicate that LASER-Agents successfully execute the scripts, achieving an average success rate (SR) of 90.48%. Most errors arise from inaccuracies in numerical comparisons, such as positions and speeds, while some mistakes result from hallucination.

Efficiency. The efficiency results are presented in Table 3, revealing an average of 1,606.09 tokens and 7.87s for generating a one-second simulation. In the Ambulance task, the inclusion of multiple dummy agents to simulate congestion significantly increased the input tokens needed to describe other vehicles' states, leading to a higher token cost. Similarly, the Reckless Driving and Sudden



Figure 5: The visualized results for script execution. (* indicate safety-critical tasks)

Jaywalker tasks require lengthy scripts for the actors, resulting in elevated input token counts. Using GPT-4o API service, generating a 40-second simulation with 3 LASER-Agents incurs a cost of approximately \$1. The majority of the time cost is attributed to

querying the LLM, with a 10-second simulation taking around one minute to generate.

Table 3: Results of effectiveness and efficiency for script execution. (* indicates safety-critical tasks)

Task	SR $\uparrow$	Token cost $\downarrow$	Time cost $\downarrow$
Accident	100%	1,556.08	9.63
Ambulance	96.43%	2,387.81	15.63
Bus	91.67%	1828.40	7.02
Reckless Driving	44.07%	2333.84	7.11
Cut-in*	98.33%	1305.83	5.76
Sudden Jaywalker*	100%	2130.40	7.55
Swerve*	100%	732.37	5.60
Three in Line 1*	93.33%	573.99	4.62
average	90.48%	1606.09	7.87

4.4 Case Study

A case study is presented to demonstrate how the LLM guides the script execution process. At each time step, LASER gathers the location, speed, and acceleration as part of the environment description and provides this information to LASER-Agents along with their respective sub-scripts. The LLM-based decision module of each LASER-Agent then reasons about the situation and makes decisions based on its script. These decisions are subsequently translated into steering and throttle control signals by a rule-based planner, which guides the agent’s movements within the simulator.

This case focuses on “rear_car” in the “Swerve” task. Following the script outlined in Figure 3, the agent maintains its speed while cruising on the highway. Once the “front_car” decelerates below 5 m/s, the agent immediately changes lanes to the right. To achieve this, we encode the sub-script and the state of the “front_car” in the user prompt and query the integrated LLM. The LLM automatically evaluates whether the termination condition (“Front_car decelerates to less than 5m/s”) has been met based on the state of the “front_car” and makes decisions according to the current action specified in the script.

We present a simplified version of the prompt process and LLM output for this case, with full details available in the repository.

System prompt. The system prompt ensures that LLM consistently follows a structured approach: receiving environment description and sub-script from user prompt, evaluating conditions, and making decisions based on the most current data (such as speed, position, and surrounding conditions). This process ultimately drives the agent’s actions in the simulated environment, with real-time decisions impacting how the scenario progresses. Every 0.5 seconds, LASER provides updated information to the LLM, enabling it to process the data and determine the agent’s next action in the sequence.

The system prompt also helps the LLM understand its role by providing specific examples within fields such as Observations. Notably, the LLM must be explicitly prompted for permission before engaging in alarmingly aggressive driving or ignoring traffic rules, ensuring controlled generation of dangerous testing scenarios.

Additionally, we hard-code the reasoning procedure to ensure that the LLM first evaluates the completion status of the previous step before determining whether it has been completed. This approach enforces sufficient reasoning before making a decision,

preventing the LLM from prematurely concluding completion and reasoning based on an incorrect outcome.

System prompt

You are a driving assistant in a simulated scene to help test autonomous driving systems. Follow the steps given by the user to generate dangerous scenes. Every 0.5s, you will be given:

-Steps: Steps to be taken to accomplish your task.

-Previous step: The step you were taking in the last 0.5s.

-Observations: The location, speed, and acceleration of you and other vehicles in the 2D plane.

You should respond to me step by step:

1. Previous Step Evaluation: Assess the completion status of the previous step based on observations and termination condition.

2. Previous Step Status: Completed/Incomplete

3. Your Current Step: The step you think should be taken next, based on current observations.

Finally: Execute actions for current frame by a tool call.

User prompt before “front_car” decelerates. This user prompt adheres to the instructions outlined in the system prompt, providing the necessary information for the LLM to reason efficiently and make accurate driving decisions. First, by setting a sequence of LCB instructions, including actions (such as “maintain 6 m/s in lane 2”) and corresponding termination conditions (such as “Slow down to less than 5 m/s in front of the vehicle”), along with the step number executed in the previous time step, the LLM is able to determine the appropriate current step in the script. This enables it to decide when to initiate a lane change based on real-time conditions.

Next, the “Observation” section provides detailed environmental data and status information, including position, speed, acceleration, and lane information.

Lane information is particularly important because it helps LLM understand the relative positions of vehicles, which is essential for making accurate decisions, especially when performing lane-change maneuvers. The data is presented with explanatory language, including definitions of longitudinal and lateral positions, helping LLM interpret the environment more effectively.

LLM reasoning before “front_car” decelerates. In this reasoning stage, the LLM evaluates the status of the previous step, verifying whether the termination condition set in the user prompt has been met. The LLM determines that the previous step is incomplete because the front_car has not yet decelerated to below 5 m/s. Based on this evaluation, the LLM confirms that the agent should continue executing Step 1, which requires maintaining a speed of 6 m/s and staying in lane 2.

LLM decision before “front_car” decelerates. Following the conclusion of reasoning, the LLM decision module generates specific behavioral decision to guide the agent in executing the corresponding actions. Since the front car has not decelerated yet, the agent should continue with its current step (Step 1), maintaining a speed of 6 m/s and staying in the current lane.

User prompt before “front_car” decelerates

Steps:

Step 1. action: Maintain 6 m/s speed in lane 2, termination_condition: Front_car decelerates to less than 5 m/s.

Step 2. action: Change from lane 2 to lane 3, termination_condition: The car is not in lane 2.

Step 3. action: Maintain 6 m/s speed in lane 3, termination_condition: Hold for next 10 seconds

Last step: Step 1.

Observation:

You are driving in lane 2 from the left on a road with 4 lanes. Position: [98.54, 3.50], speed: 5.84 m/s, acceleration: 0.73 m/s^2 , lane position: 98.54 m.

- ‘Front_car’. Position: [108.83, 3.50], speed: 5.81 m/s, acceleration: 1.02 m/s^2 , lane position: 108.83 m.

- ‘VUT’ (lane 3, to your right). Position: [89.27, 7.00], speed: 7.88 m/s, acceleration: 3.73 m/s^2 , lane position: 89.27 m.

User prompt after “front_car” decelerates

Steps:

Step 1. action: Maintain 6 m/s speed in lane 2, termination_condition: Front_car decelerates to less than 5 m/s.

Step 2. action: change from lane 2 to lane 3, termination_condition: The car is not in lane 2.

Step 3. action: Maintain 6 m/s speed in lane 3, termination_condition: Hold for the next 10 seconds.

Last step: Step 1.

Observation:

You are driving in lane 2 from the left on a road with 4 lanes. Position: [104.42, 3.50], speed: 5.76 m/s, acceleration: 0.70 m/s^2 , lane position: 104.42 m.

- ‘Front_car’. Position: [114.07, 3.50], speed: 4.65 m/s, acceleration: 1.24 m/s^2 , lane position: 114.07 m.

- ‘VUT’ (lane 3, to your right). Position: [97.10, 7.02], speed: 7.99 m/s, acceleration: 0.65 m/s^2 , lane position: 97.10 m.

LLM reasoning before “front_car” decelerates

1. Previous Step Evaluation: We are maintaining a speed close to 6 m/s in lane 2. The front car in the same lane is driving at 5.81 m/s and accelerating.

2. Previous Step Status: Incomplete

3. Current Step: Step 1. Maintain 6 m/s speed in lane 2.

Executing actions for the current frame:

- Maintain a target speed of 6 m/s

- Follow the current lane (lane 2)

Let’s proceed with these actions.

LLM reasoning after “front_car” decelerates

1. Previous Step Evaluation: The previous step is Step 1. Your current speed is 5.76 m/s and front_car’s speed is 4.65 m/s. The front_car’s speed is less than 5 m/s, which meets the termination condition.

2. Previous Step Status: Completed

3. Current Step: Step 2. Change from lane 2 to lane 3.

Executing actions for the current frame:

- Current Step Number: 2

- Lane Change Direction: Right Lane Change

- Lane Change Delay: 0.1 s

- Target Speed: Maintain 6 m/s

Executing the action.

LLM decision before “front_car” decelerates

current_step_number: 1

lane_change_direction: FOLLOW LANE

lane_change_delay: 0

target_speed: 6

User prompt after “front_car” decelerates. This User prompt presents updated environmental information at a new time stamp after the “front_car” decelerates, formatted similarly to the user prompt before the front car decelerated. At this moment, the last step of the agent is still Step 1, and the current speed of the front car is now 4.65 m/s.

LLM reasoning after “front_car” decelerates. This reasoning section occurs after the “front_car” decelerates. At the new time step, the LLM reasoning stage reassesses whether the previous step has been completed. The observations confirm that the “front_car” has decelerated below 5 m/s, fulfilling the termination condition for Step 1. Therefore, the status of Step 1 is marked as “Completed.” The action to execute the lane change is initiated accordingly.

LLM decision after “front_car” decelerates. Following the reasoning, the LLM decision module generates the corresponding

LLM decision after “front_car” decelerates

current_step_number: 2

lane_change_direction: RIGHT LANE CHANGE

lane_change_delay: 0.1

target_speed: 6

behavioral decision. With the front car’s deceleration confirmed, the LLM decides to execute Step 2. The agent is then instructed to change lanes to the right with a slight delay of 0.1 seconds and continue at a speed of 6 m/s.

5 Limitations

Manual description of map layout. LASER’s execution relies on manually formatting map layouts for scenario generation. This approach can lead to inaccuracies and inefficiencies, particularly

in complex environments. Implementing automated map interpretation (e.g., querying image-to-text models with the initial frame) could greatly enhance the framework’s scalability and accuracy.

Lack of automatic search for scenario details. The current system necessitates user-in-the-loop revision for scripts. While this allows tailoring the generation that aligns with users’ intentions, it restricts the system’s capacity to autonomously generate a multitude of test cases with the same initial goals but varying details. Developing a more intelligent script writer capable of automatically searching for reasonable and elaborate scenario details poses a challenge due to the knowledge gap between off-the-shelf LLMs and the specific requirements of the simulation environment.

Computational overhead for real-time execution. Integrating LLM-controlled agents with real-time execution in complex environments incurs significant computational overhead, especially when scaling the simulation to multiple agents. Future enhancements could focus on optimizing interactions between the LLM-based decision-making module and the rule-based planner, aiming to reduce latency and computational load while maintaining high performance and decision accuracy.

Generalization to real-world scenarios. Although our framework demonstrates strong performance in simulated environments, its ability to generalize to real-world driving scenarios may hinge on the simulation’s fidelity. Ensuring that virtual agents accurately mimic human driver behavior across diverse global contexts remains an ongoing challenge.

6 Conclusion

In this paper, we introduce LASER, a novel approach that leverages LLMs to generate on-demand traffic simulations. Our two-stage framework separates scenario generation from real-time execution, providing greater flexibility, scalability, and customizability compared to traditional simulation methods. By utilizing LLM-controlled agents, LASER offers a more human-like interpretation of driving behaviors, ensuring coherent and realistic interactions within the simulated environment. The experimental results demonstrate that LASER effectively meets diverse user requirements for both general and safety-critical driving scenarios, showcasing high accuracy and adaptability in scenario creation. Overall, the proposed approach represents a significant advancement in on-demand traffic simulation for ADS training and testing.

Acknowledgment

We are thankful to the anonymous reviewers for their helpful comments. This work is supported by Frontier Technologies R&D Program of Jiangsu (#BF2024059), the National Natural Science Foundation of China (Grants #62025202, #62172199), and the Collaborative Innovation Center of Novel Software Technology and Industrialization. Jingwei Xu (jingwei@nju.edu.cn) is the corresponding author.

References

- [1] Suneel Belkhale, Tianli Ding, Ted Xiao, Pierre Sermanet, Quon Vuong, Jonathan Tompson, Yevgen Chebotar, Debidatta Dwibedi, and Dorsa Sadigh. 2024. Rt-h: Action hierarchies using language. *arXiv preprint arXiv:2403.01823* (2024).
- [2] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. 2023. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818* (2023).
- [3] Holger Caesar, Varun Bankiti, Alex H Lang, Sourabh Vora, Venice Erin Liong, Qiang Xu, Anush Krishnan, Yu Pan, Giancarlo Baldan, and Oscar Beijbom. 2020. nuscenes: A multimodal dataset for autonomous driving. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 11621–11631.
- [4] Holger Caesar, Juraj Kabzan, Kok Seang Tan, Whye Kit Fong, Eric Wolff, Alex Lang, Luke Fletcher, Oscar Beijbom, and Sammy Omari. 2021. nuplan: A closed-loop ml-based planning benchmark for autonomous vehicles. *arXiv preprint arXiv:2106.11810* (2021).
- [5] Anh-Quan Cao, Angela Dai, and Raoul de Charette. 2024. Pasco: Urban 3d panoptic scene completion with uncertainty awareness. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 14554–14564.
- [6] Jordi Casas, Jaime L Ferrer, David Garcia, Josep Perarnau, and Alex Torday. 2010. Traffic simulation with aimsun. *Fundamentals of traffic simulation* (2010), 173–232.
- [7] Di Chen, Meixin Zhu, Hao Yang, Xuesong Wang, and Yinhai Wang. 2024. Data-driven Traffic Simulation: A Comprehensive Review. *IEEE Transactions on Intelligent Vehicles* (2024).
- [8] Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. 2017. CARLA: An open urban driving simulator. In *Conference on robot learning*. PMLR, 1–16.
- [9] Danny Driess, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayyaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. 2023. Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378* (2023).
- [10] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [11] Martin Fellendorf and Peter Vortisch. 2010. Microscopic traffic flow simulator VISSIM. *Fundamentals of traffic simulation* (2010), 63–93.
- [12] Shuo Feng, Xintao Yan, Haowei Sun, Yiheng Feng, and Henry X Liu. 2021. Intelligent driving intelligence test for autonomous vehicles with naturalistic and adversarial environment. *Nature communications* 12, 1 (2021), 748.
- [13] Xunjiang Gu, Guanyu Song, Igor Gilitschenski, Marco Pavone, and Boris Ivanovic. 2024. Producing and leveraging online map uncertainty in trajectory prediction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 14521–14530.
- [14] S Huang, L Dong, W Wang, Y Hao, S Singhal, S Ma, T Lv, L Cui, OK Mohammed, B Patra, et al. 2023. Language is not all you need: aligning perception with language models (2023). *arXiv preprint arXiv:2302.14045* (2023).
- [15] Yuanhui Huang, Wenzhao Zheng, Yungpeng Zhang, Jie Zhou, and Jiwen Lu. 2023. Tri-perspective view for vision-based 3d semantic occupancy prediction. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 9223–9232.
- [16] Haoyi Jiang, Tianheng Cheng, Naiyu Gao, Haoyang Zhang, Tianwei Lin, Wenyu Liu, and Xinggang Wang. 2024. Symphonize 3d semantic scene completion with contextual instance queries. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 20258–20267.
- [17] Yiming Li, Zhiding Yu, Christopher Choy, Chaowei Xiao, Jose M Alvarez, Sanja Fidler, Chen Feng, and Anima Anandkumar. 2023. Voxformer: Sparse voxel transformer for camera-based 3d semantic scene completion. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 9087–9098.
- [18] Pablo Alvarez Lopez, Michael Behrisch, Laura Bieker-Walz, Jakob Erdmann, Yun-Pang Flötteröd, Robert Hilbrich, Leonhard Lücken, Johannes Rummel, Peter Wagner, and Evamarie Wießner. 2018. Microscopic traffic simulation using sumo. In *2018 21st international conference on intelligent transportation systems (ITSC)*. IEEE, 2575–2582.
- [19] OpenAI. 2024. Introducing OpenAI o1-preview. <https://openai.com/index/introducing-openai-o1-preview/>.
- [20] Daehee Park, Jaeseok Jeong, Sung-Hoon Yoon, Jaewoo Jeong, and Kuk-Jin Yoon. 2024. T4P: Test-Time Training of Trajectory Prediction via Masked Autoencoder and Actor-specific Token Memory. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 15065–15076.
- [21] Katrin Renz, Long Chen, Ana-Maria Marcu, Jan Hünemann, Benoit Hanotte, Alice Karnsund, Jamie Shotton, Elahe Arani, and Oleg Sinavski. 2024. CarLLaVA: Vision language models for camera-only closed-loop driving. *arXiv preprint arXiv:2406.10165* (2024).
- [22] Tim Salzmann, Boris Ivanovic, Punarjay Chakravarty, and Marco Pavone. 2020. Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK,*

- August 23–28, 2020, *Proceedings, Part XVIII 16*. Springer, 683–700.
- [23] Hao Shao, Yuxuan Hu, Letian Wang, Guanglu Song, Steven L. Waslander, Yu Liu, and Hongsheng Li. 2024. Lmdrive: Closed-loop end-to-end driving with large language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 15120–15130.
 - [24] Hao Shao, Letian Wang, Ruobing Chen, Steven L. Waslander, Hongsheng Li, and Yu Liu. 2023. Reasonnet: End-to-end driving with temporal and global reasoning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 13723–13733.
 - [25] SP Sharan, Francesco Pittaluga, Manmohan Chandraker, et al. 2023. Llm-assist: Enhancing closed-loop planning with language-based reasoning. *arXiv preprint arXiv:2401.00125* (2023).
 - [26] Simon Suo, Sebastian Regalado, Sergio Casas, and Raquel Urtasun. 2021. Traffic-sim: Learning to simulate realistic multi-agent behaviors. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 10400–10409.
 - [27] Tesla. 2023. 2023 Investor Day | Tesla. <https://www.youtube.com/watch?v=H1zEzVUV7w>. Accessed: 2024-09-13.
 - [28] Tesla. 2024. Full Self-Driving (Supervised) | Tesla. <https://www.youtube.com/watch?v=TUDiG7PcLBs>. Accessed: 2024-09-13.
 - [29] Chalavadi Vishnu, Vineel Abhinav, Debadya Roy, C Krishna Mohan, and Ch Sohan Babu. 2023. Improving multi-agent trajectory prediction using traffic states on interactive driving scenarios. *IEEE Robotics and Automation Letters* 8, 5 (2023), 2708–2715.
 - [30] Wenhai Wang, Jiangwei Xie, ChuanYang Hu, Haoming Zou, Jianan Fan, Wenwen Tong, Yang Wen, Silei Wu, Hanming Deng, Zhiqi Li, et al. 2023. Drivemlm: Aligning multi-modal large language models with behavioral planning states for autonomous driving. *arXiv preprint arXiv:2312.09245* (2023).
 - [31] Xiaofeng Wang, Zheng Zhu, Guan Huang, Xinze Chen, Jiagang Zhu, and Jiwen Lu. 2023. Drivedreamer: Towards real-world-driven world models for autonomous driving. *arXiv preprint arXiv:2309.09777* (2023).
 - [32] Yuqi Wang, Jiawei He, Lue Fan, Hongxin Li, Yuntao Chen, and Zhaoxiang Zhang. 2024. Driving into the future: Multiview visual forecasting and planning with world model for autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 14749–14759.
 - [33] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *arXiv:2201.11903 [cs.CL]* <https://arxiv.org/abs/2201.11903>
 - [34] Yuxi Wei, Zi Wang, Yifan Lu, Chenxin Xu, Changxing Liu, Hao Zhao, Siheng Chen, and Yanfeng Wang. 2024. Editable scene simulation for autonomous driving via collaborative llm-agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 15077–15087.
 - [35] Yi Wei, Linqing Zhao, Wenzhao Zheng, Zheng Zhu, Jie Zhou, and Jiwen Lu. 2023. Surroundocc: Multi-camera 3d occupancy prediction for autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 21729–21740.
 - [36] Licheng Wen, Daocheng Fu, Xin Li, Xinyu Cai, Tao Ma, Pinlong Cai, Min Dou, Botian Shi, Liang He, and Yu Qiao. 2023. Dilu: A knowledge-driven approach to autonomous driving with large language models. *arXiv preprint arXiv:2309.16292* (2023).
 - [37] Yuqing Wen, Yucheng Zhao, Yingfei Liu, Fan Jia, Yanhui Wang, Chong Luo, Chi Zhang, Tiancai Wang, Xiaoyan Sun, and Xiangyu Zhang. 2024. Panacea: Panoramic and controllable video generation for autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 6902–6912.
 - [38] Wikipedia. 2018. ISO 26262 – Road vehicles – Functional safety. https://en.wikipedia.org/wiki/ISO_26262.
 - [39] Wikipedia. 2023. Proportional–integral–derivative controller – Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/Proportional%E2%80%93integral%E2%80%93derivative_controller
 - [40] Danfei Xu, Yuxiao Chen, Boris Ivanovic, and Marco Pavone. 2023. Bits: Bi-level imitation for traffic simulation. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2929–2936.
 - [41] Jiawei Zhang, Chejian Xu, and Bo Li. 2024. ChatScene: Knowledge-Enabled Safety-Critical Scenario Generation for Autonomous Vehicles. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 15459–15469.
 - [42] Linrui Zhang, Zhenghao Peng, Quanyi Li, and Bolei Zhou. 2023. Cat: Closed-loop adversarial training for safe end-to-end driving. In *Conference on Robot Learning*. PMLR, 2357–2372.
 - [43] Guosheng Zhao, Xiaofeng Wang, Zheng Zhu, Xinze Chen, Guan Huang, Xiaoyi Bao, and Xingang Wang. 2024. Drivedreamer-2: Llm-enhanced world models for diverse driving video generation. *arXiv preprint arXiv:2403.06845* (2024).